TurboQuant

Redefining AI Efficiency with Extreme Compression

Google Research · ICLR 2026

Siyu Hu, Binghui Ni

The Context Problem: RAG vs. Long Context

RAG

Retrieval-Augmented Generation

✅ Cheap to run — only retrieves what it needs
✅ Scales to huge document libraries

❌ Can miss context across many documents
❌ Extra pipeline complexity
❌ Retrieval errors hurt quality

LONG CONTEXT ★ Preferred

Feed everything into the model at once

✅ Sees the full picture — no missed context
✅ Better reasoning across long documents
✅ Simpler pipeline, no retrieval step

❌ Slow & expensive as length grows
❌ Huge memory usage (KV Cache!)

The problem: Long Context hits a memory wall called the KV Cache Bottleneck

Why Long Context Is Expensive: The KV Cache

Each Token Processed
word / piece of text

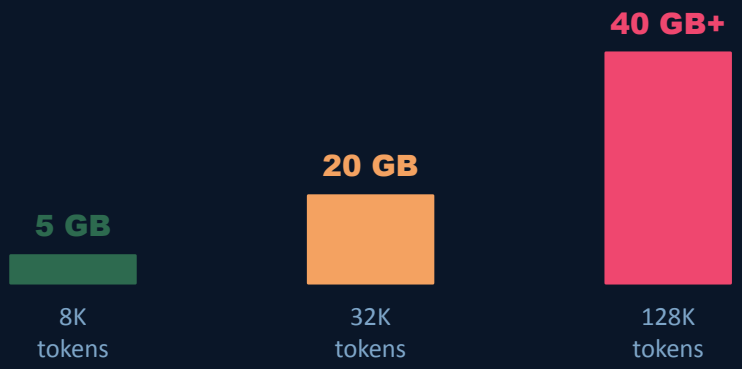
→

Keys & Values Stored in Memory
KV Cache (FP16)

→

Memory Explodes
128K tokens = 40 GB+

KV Cache Memory vs. Context Length (Llama 70B, FP16)



Why This Hurts

🐢 More tokens ⟶ more KV data to read per generation step ⟶ slower output

💸 More memory needed ⟶ need bigger (more expensive) GPUs

🚧 Memory bandwidth, not compute, becomes the bottleneck

Introducing TurboQuant

Google Research's answer to the KV Cache bottleneck

6×
Less KV Cache Memory

8×
Faster Attention (H100)

3 bits
vs 16 bits FP16

0%
Accuracy Loss

✅ No retraining, no fine-tuning, no calibration data — works on any model out-of-the-box

What is Quantization?

Numbers in a computer use 16 bits of storage (FP16).

Quantization means storing the same number with fewer bits — like rounding 3.14159 to 3.14.

TurboQuant does this without losing accuracy.

How TurboQuant Works — Step 1: PolarQuant

STAGE 1 OF 2

The Problem with Naive Quantization

KV vectors have 'outlier' coordinates — a few values carry most of the information. Naively compressing them destroys those important values.

↓ PolarQuant's Fix

1. **Random Rotation:** Apply a random rotation to each vector. This spreads all values evenly — no more outlier coordinates.
2. **Polar Coordinates:** Convert the vector from Cartesian (x,y,z…) into polar form (radius + angles). Removes normalization overhead.
3. **Optimal Quantization:** Distribution is now predictable, so use the mathematically best quantizer (Lloyd-Max) — no calibration data needed.

Analogy: The Uneven Bag of Rocks 🪨

Imagine you have a bag of rocks — a few giant ones and many tiny ones.

If you compress the bag naively, you crush the tiny rocks but the big ones won't fit — you lose them.

PolarQuant first breaks the big rocks apart so everything is the same size (random rotation).

Now you can compress evenly — nothing gets crushed and nothing is lost.

How TurboQuant Works — Step 2: QJL Error Correction

STAGE 2 OF 2

Quantized Johnson-Lindenstrauss (QJL)

Even after PolarQuant, tiny rounding errors remain.

These errors are small but they can add up when the model computes attention scores — the step that decides which tokens to pay attention to.

QJL adds just 1 extra bit per coordinate to store a 'correction signal' that removes this bias — keeping attention scores accurate.

📐 Analogy: Spell-Check for Numbers — catches small errors before they cause problems.

Full Pipeline

① **KV vector (FP16)**
16 bits per number

↓

② **PolarQuant**
Rotate → polar → quantize

↓

③ **QJL (+1 bit)**
Error correction

↓

④ **Result: ~3 bits**
6× smaller, same accuracy

Results: Benchmark Performance

✅ TurboQuant matches full FP16 accuracy across ALL benchmarks while using 6× less memory

Benchmark Results

Benchmark	What It Tests	FP16 (baseline)	TurboQuant 3.5-bit
LongBench	QA, code, summarization	50.06	**50.06** ✅
Needle-in-Haystack	Find 1 fact in 100K tokens	0.997	**0.997** ✅
RULER, L-Eval, SCROLLS	Long-context reasoning	—	**Perfect** ✅

vs. KIVI (previous best):

Compression:	KIVI: 2.6×	**TurboQuant: 6×**
Bit-width:	KIVI: 2-bit	**TurboQuant: 3-bit (better quality)**
Calibration needed?:	KIVI: Yes	**TurboQuant: No**
Accuracy parity?:	KIVI: Partial	**TurboQuant: Full**

Real-World Impact & What Comes Next

Much Lower Cost

On 2× H100s, serving 128K-context requests could drop from ~$2,088/user/month to ~$380/user/month — a 5× cost reduction.

More Users per GPU

A KV cache needing 40 GB shrinks to ~7 GB. The same GPU can now serve 6× more users at once.

Vector Search Too

TurboQuant also compresses embedding databases for nearest-neighbor search — with near-zero indexing time vs. hours for competitors.

What Could This Enable?

- Phones running 1M-token context models entirely on-device
- Enterprise AI at a fraction of current cloud costs
- Real-time analysis of entire codebases or legal documents in one go
- Democratized long-context AI — not just for organizations with big GPU budgets

Key Takeaways

1. Long context is better than RAG — but was too expensive
2. The KV Cache is why: it grows linearly with context length, eating GPU memory
3. TurboQuant compresses KV Cache 6× — from 16 bits to 3 bits — with zero accuracy loss
4. PolarQuant: random rotation + polar coords eliminates outliers and normalization overhead
5. QJL: 1-bit error correction keeps attention scores precise
6. No retraining. Plug-and-play. Works on any transformer model.

Thank you! | Source: research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/